UNIVERSIDAD TECNOLÓGICA

TULA-TEPEJI

Tecnologías de la Información

Desarrollo De Software Multiplataforma

Estructura de datos

Investigación tipos de datos abstractos

Desarrollado por:

- Adair Jesús Cruz Huitrón 25300767 G1

Edith Campos Cruz

A 27 de Septiembre, 2026

1. Investigación

En arreglos

- Burbuja (Bubble Sort)

  El método de burbuja funciona cuando se recorre el arreglo, comparando un numero y su posterior, si es menor, se intercambian de lugares, asi hasta que todos los números estén ordenados. Su estrategia se basa sólo en comparaciones e intercambios sucesivos

  Complejidad temporal:

Caso	Complejidad
Mejor	O(n)
Promedio	$O(n^2)$
Peor	$O(n^2)$

  Es un método estable, pero no pensado para grandes cantidades de datos.

  Su ventaja principal es que es muy fácil de entender y codificar

  Su mayor desventaja es que la ordenación es muy lenta, comparandola con otros métodos.

  Es muy buen método de ordenamiento para comprender los métodos, o para listas muy pequeñas, sacrificando rendimiento por simplicidad.

- Selección (Selection Sort)

  El método de selection divide el arreglo en dos partes, uno donde ya está ordenado el arreglo y la otra parte aún no. Lo que hace es recorrer la lista para identificar si el actual es menor que el anterior, de ser el caso intercambia lugares y sigue comparando si el actual en su nueva posición es menor que el anterior, si sí intercambia lugares y continua hasta que se rompa la condición

Complejidad temporal:

Caso	Complejidad
Mejor	O(n)
Promedio	$O(n^2)$
Peor	$O(n^2)$

Es un método no estable, ya que puede cambiar el orden relativo de los elementos iguales. No pensado para grandes cantidades de datos, ya que tiene complejidad $O(n^2)$

Su ventaja principal es que es fácil de entender e implementar, además de requerir únicamente un espacio de memoria adicional constante de O(1).

Su mayor desventaja es que no mantiene el orden relativo de los elementos iguales, lo que significa que no es estable.

Es muy buen método de ordenamiento para comprender los métodos, o para listas muy pequeñas, donde la memoria es escaza

- Inserción (Insertion Sort)

  El método de inserción divide el arreglo en dos partes, uno donde ya está ordenado el arreglo y la otra parte aún no. Lo que hace es recorrer la lista para identificar al menor y cuando llega al final intercambia la posición del menor con la posición de elementos que ya están ordenados

Complejidad temporal:

Caso	Complejidad
Mejor	O(n)
Promedio	$O(n^2)$
Peor	$O(n^2)$

Es un método estable. No pensado para grandes cantidades de datos, ya que tiene complejidad $O(n^2)$

Su ventaja principal es que esácil de entender e implementar, es eficiente al momento de asignar espacio, además de que su caso óptimo es un O(n)

Su mayor desventaja es que no está pensado para listas con grandes cantidades de datos, además de ser menos eficiente que otros

Es muy buen método de ordenamiento para comprender los métodos, o para listas muy pequeñas, donde la memoria es escaza

- Merge Sort

  El método merge lo que hace es dividir en partes iguales un arreglo, asi hasta tener solo de un valor, y posteriormente ir ordenando y juntando pedazos de 2 en 2 hasta ordenar 2 mitades finales del arreglo

  Complejidad temporal:

Caso	Complejidad
Mejor	O(n log n)
Promedio	O(n log n)
Peor	O(n log n)

  Es un método estable, mantiene el orden relativo de los elementos.

  Su ventaja principal es que tiene un rendimiento óptimo para hasta el peor de los casos, y su enfoque de dividir y vencer es sencillo.

  Su mayor desventaja es que necesita memoria adicional, además de ser más lento que el quick sort

- Quick Sort

  El método quick aplica la misma lógica de dividir y vencer, pero de manera distinta.

  Primer se selecciona un elemento llamado pivote, después organiza los elementos a su izquierda menores a el y a la derecha mayores a el, posteriormente se hace recursivo y se hace lo mismo de cada lado

Complejidad temporal:

Caso	Complejidad
Mejor	O(n log n)
Promedio	O(n log n)
Peor	$O(n^2)$

No es un método estable.

Está pensado para grandes cantidades de elementos a ordenar

Su ventaja principal es que tiene un rendimiento óptimo para cantidades masivas de datos, además de poca memoria para funcionar.

Su mayor desventaja es que no es óptimo para casos con pocos elementos o si el pivote no se elige bien.

Ideal cuando se necesita velocidad, sacrificando la estabilidad

- Heap Sort

  Un Heap es una estructura que puede representarse como un árbol, aunque puede almacenarse directamente en un arreglo.

  Lo que hace es construir un heap, para luego eliminar el elemento en la cima, poniendolo al final del arreglo, ordenando nuevamente y haciendo recursivo la acción.

  Complejidad temporal:

Caso	Complejidad
Mejor	O(n log n)
Promedio	O(n log n)
Peor	O(n log n)

No es un método estable.

Está pensado para grandes cantidades de elementos a ordenar

Su ventaja principal es que tiene un rendimiento equitativo para todas las situaciones, además de trabajar con el mínimo de memoria.

Su mayor desventaja es su O(n log n) crece a medida de los términos además de ser ineficiente por esto mismoo.

Ideal cuando se necesita velocidad, sacrificando la estabilidad

- Ordenación mediante Arrays.sort() de Java

Ordena los elementos según el orden natural o un comparador personalizado.

Funciona con implementaciones de listas, no con matrices.

Modifica la lista directamente sin crear una nueva lista.

Complejidad temporal:

Caso	Complejidad
Mejor	O(n log n)
Promedio	O(n log n)
Peor	O(n log n)

Es un método estable

pensado para grandes cantidades más que nada

Tabla de datos

Algoritmo	Mejor	Promedio	Peor	Espacio	Estable	Grandes datos
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	si	no
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	no	no
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	si	no
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	si	si
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	no	si
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	no	si
Arrays.sort()	depende	depende	depende	depende	depende	si

Parte D. Análisis

¿Qué método realizó menos comparaciones con el conjunto de datos utilizado?

Quick sort

¿El método que realizó menos comparaciones necesariamente es el mejor método de ordenación? Explica.

Si, ya que quick sort es ampliamente utilizado por su rendimiento de tiempo, así como el ahorro de memoria al ser recursivo

¿Qué método elegirías para ordenar un arreglo pequeño? ¿Por qué?

Insertion, ya que es para mí el más fácil de codificar

¿Qué método elegirías para trabajar con una gran cantidad de datos? Yo elegiría merge sort ya que es uno de los más famosos por su complejidad O(n log n) para todos los casos posibles

¿Cuál es la diferencia entre el comportamiento teórico de un algoritmo y su comportamiento observado en una ejecución?

Es a veces muy apegado a casos, pero no se pueden diferenciar así a simple vista con tan pocas ejecuciones y tan pocos datos (en comparación con miles)

¿Qué ventajas ofrece utilizar Arrays.sort() en lugar de implementar manualmente un algoritmo de ordenación?

Muchas, entre ellas:

- Menos código: al no tener que declarar todo el código de un método de ordenamiento
- Funciona con diferentes tipos de datos.
- utiliza implementaciones optimizadas según el tipo de arreglo

¿Por qué es importante conocer los algoritmos de ordenación si Java ya proporciona métodos para ordenar arreglos?

Por que es necesario comprender como funcionan estos métodos, teniendo consciencia sobre por qué funciona una solución, cuánto cuesta computacionalmente y cuándo puede ser conveniente utilizar otra estrategia. Es como usar una calculadora sabiendo matemáticas, las operaciones se podrian hacer manualmente pero seria demasiado tardado.

Codigos java:

bubble_sort.java

```
package Unidad_1.Metodos_ordenacion;


public class bubble_sort {
public static void bubbleSort(int[] numeros) {
System.out.println("Intercambios/comparaciones: ");
for (int i = 0; i < numeros.length - 1; i++) {
for (int j = 0; j < numeros.length - 1 - i; j++) {
if (numeros[j] > numeros[j + 1]) {
System.out.print("+");
int temporal = numeros[j];
numeros[j] = numeros[j + 1];
```

```
numeros[j + 1] = temporal;
}
}
}
System.out.println();
System.out.print("[");
for(int i=0 ;i<numeros.length;i++){
System.out.print(numeros[i]+",");
}
System.out.print("]");
}
}
```

quick_sort.java:

```
package Unidad_1.Metodos_ordenacion;

public class heap_sort {
public void sort(int arr[])
int n = arr.length;
// Build heap (rearrange array)
for (int i = n / 2 - 1; i >= 0; i--)
heapify(arr, n, i);
// One by one extract an element from heap
for (int i = n - 1; i >= 0; i--) {
// Move current root to end
int temp = arr[0];
arr[0] = arr[i];
arr[i] = temp;
// call max heapify on the reduced heap
heapify(arr, i, 0);
}
}
// To heapify a subtree rooted with node i which is
// an index in arr[]. n is size of heap
public void heapify(int arr[], int n, int i)
{
int largest = i; // Initialize largest as root
int l = 2 * i + 1; // left = 2*i + 1
int r = 2 * i + 2; // right = 2*i + 2

// If left child is larger than root
if (l < n && arr[l] > arr[largest])
largest = l;
// If right child is larger than largest so far
if (r < n && arr[r] > arr[largest])
```

```
largest = r;
// If largest is not root
if (largest != i) {
System.out.print("+");
int swap = arr[i];
arr[i] = arr[largest];
arr[largest] = swap;
// Recursively heapify the affected sub-tree
heapify(arr, n, largest);
}
}
public static void imprimirArreglo(int[] arr) {
for (int i = 0; i < arr.length; i++) {
System.out.print(arr[i] + " ");
}
System.out.println();
}
}
```

heap_sort.java

```
package Unidad_1.Metodos_ordenacion;

public class quick_sort {
// partition function
public static int partition(int[] arr, int low, int high) {
// choose the pivot
int pivot = arr[high];
// index of smaller element and indicates
// the right position of pivot found so far
int i = low - 1;

// traverse arr[low..high] and move all smaller
// elements to the left side. Elements from low to
// i are smaller after every iteration
for (int j = low; j <= high - 1; j++) {
if (arr[j] < pivot) {
i++;
swap(arr, i, j);
}
}
// Move pivot after smaller elements and
// return its position
swap(arr, i + 1, high);
return i + 1;
}
```

```
// swap function
public static void swap(int[] arr, int i, int j) {
System.out.print("+");
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
}
// the QuickSort function implementation
public void quickSort(int[] arr, int low, int high) {
if (low < high) {
// pi is the partition return index of pivot
int pi = partition(arr, low, high);

// recursion calls for smaller elements
// and greater or equals elements
quickSort(arr, low, pi - 1);
quickSort(arr, pi + 1, high);
}
}
public static void imprimirArreglo(int[] arr) {
for (int i = 0; i < arr.length; i++) {
System.out.print(arr[i] + " ");
}
System.out.println();
}
}
```

main.java

```
package Unidad_1.Metodos_ordenacion;

import java.util.Arrays;

import javax.swing.JOptionPane;

public class main {
public static void main(String[] args) {
int n = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el numero de elementos de la
matriz"));
int[] x = new int [n];
for(int i=0; i<x.length;i++){
x[i] = Integer.parseInt(JOptionPane.showInputDialog(("Ingrese el numero:" +(i+1))));
}
System.out.println("Arreglo original");
for(int t = 0; t<n ; t++){
```

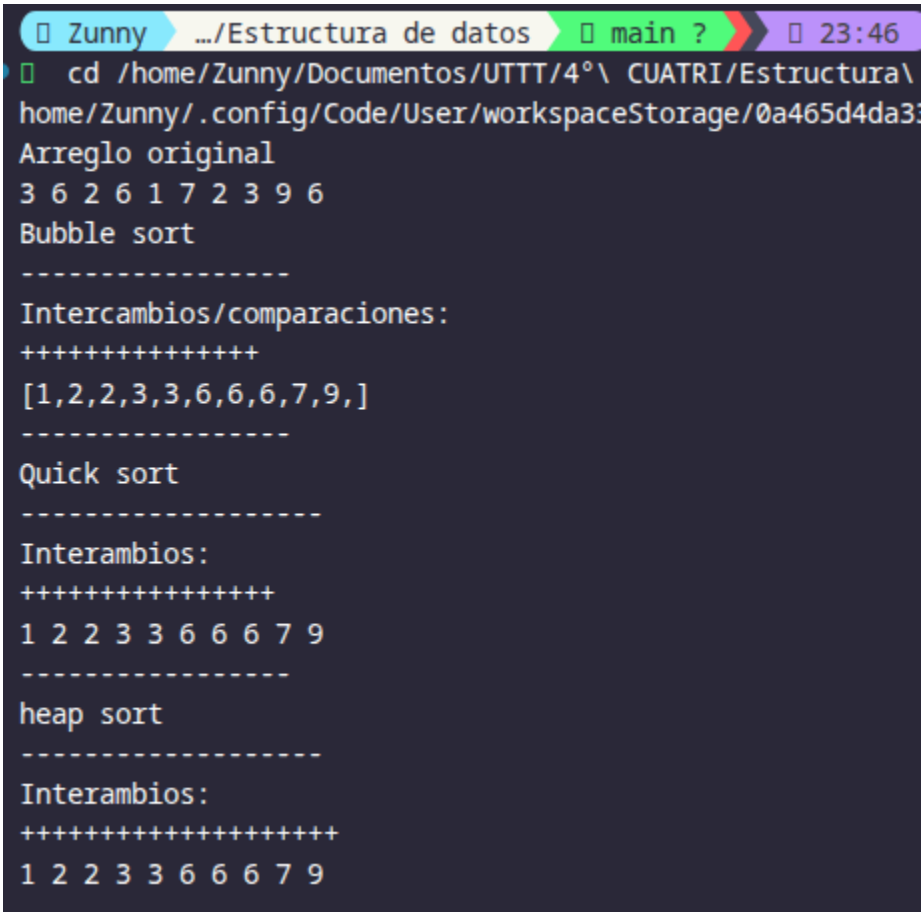
```
        System.out.print(x[t]+" ");
        }
        int[] copia = Arrays.copyOf(x, x.length);

        System.out.println("\nBubble sort\n----------------");
        bubble_sort bubble = new bubble_sort();
        bubble.bubbleSort(copia);
        System.out.println("\n----------------");


        System.out.println("Quick sort\n----------------");
        quick_sort quick = new quick_sort();
        copia = Arrays.copyOf(x, x.length);
        System.out.print("Interambios: \n");
        quick.quickSort(copia, 0, n-1);
        System.out.println();
        quick.imprimirArreglo(copia);
        System.out.println("----------------");


        System.out.println("heap sort\n----------------");
        copia = Arrays.copyOf(x, x.length);
        heap_sort heap = new heap_sort();
        System.out.print("Interambios: \n");
        heap.sort(copia);
        System.out.println();
        heap.imprimirArreglo(copia);
    }
}
```

Evidencia:



Bibliografía:

GeeksforGeeks. (2025, July 23). *Time and Space complexity Analysis of bubble sort*. GeeksforGeeks. https://www-geeksforgeeks-org.translate.goog/dsa/time-and-space-complexity-analysis-of-bubble-sort/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc&_x_tr_hist=true

GeeksforGeeks. (2026, September 27). *Time complexity and space analysis of insertion sort*. GeeksforGeeks. https://www-geeksforgeeks-org.translate.goog/dsa/time-and-space-complexity-of-insertion-sort-algorithm/

GeeksforGeeks. (2026, September 27). *Time complexity and space analysis of insertion sort*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-insertion-sort-algorithm/

GeeksforGeeks. (2026, September 25). *Quick sort*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/

GeeksforGeeks. (2026, February 5). *Heap sort*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/heap-sort/

Dotnet-Bot. (n.d.). *Arrays.Sort método (Java.Util)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/api/java.util.arrays.sort?view=net-android-34.0

GeeksforGeeks. (2025, December 8). *Selection Sort*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/